

算法分析第1次作业

1-1

1-1 求下列函数的渐近表达式：

$$3n^2 + 10n;$$

$$n^2/10 + 2^n;$$

$$21 + 1/n;$$

$$\log n^3;$$

$$10 \log 3^n$$

第1-1题答案:

(1) $3n^2 + 10n$

渐近表达式为 $O(n^2)$

(2) $\frac{n^2}{10} + 2^n$

渐近表达式为 $O(2^n)$

(3) $21 + \frac{1}{n}$

渐近表达式为 $O(1)$

(4) $\log n^3$

$$= 3 \log n$$

渐近表达式为 $O(\log n)$

(5) $10 \log 3^n$

$$= 10n \log 3$$

渐近表达式为 $O(n)$

1-2

1-2 试论 $O(1)$ 和 $O(2)$ 的区别。

第1-2题答案:

在算法的时间复杂度分析中, $O(1)$ 与 $O(2)$ 没有本质区别, 它们都是常数阶复杂度。

原因如下:

- 1. 等价性:** 根据大O阶的定义, 大O符号表示的是函数增长率的上界。在分析时会忽略常数系数。因为 2 是常数, 所以 $O(2) = O(1 \times 2) = O(1)$ 。
- 2. 共同特征:** 两者都表示算法的执行时间不随输入规模 n 的增加而增长, 即无论处理的数据量有多大, 执行时间都是恒定的。

1-3

1-3 按照渐近阶从低到高的顺序排列以下表达式: $4n^2$, $\log n$, 3^n , $20n$, 2 , $n^{2/3}$ 。又 $n!$ 应该排在哪一位?

第1-3题答案:

$4n^2$ 的渐进表达式为 $O(n^2)$

$\log n$ 的渐进表达式为 $O(\log n)$

3^n 的渐进表达式为 $O(3^n)$

$20n$ 的渐进表达式为 $O(n)$

2 的渐进表达式为 $O(1)$

$n^{2/3}$ 的渐进表达式为 $O(n^{2/3})$

$n!$ 的渐进表达式为 $O(n!)$

排序如下:

$$2 < \log n < n^{2/3} < 20n < 4n^2 < 3^n < n!$$

$n!$ 应该排在最高位, 因为根据斯特林公式 $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$ 。当 n 足够大时, 底数 n/e 会远大于 3, 因此 $n!$ 会迅速超过 3^n 。

1-4

1-4 (1) 假设某算法在输入规模为 n 时的计算时间为 $T(n)=3 \times 2^n$ 。在某台计算机上实现并完成该算法的时间为 t 秒。现有另一台计算机，其运行速度为第一台的 64 倍，那么在这台新机器上用同一算法在 t 秒内能解输入规模为多大的问题？

(2) 若上述算法的计算时间改进为 $T(n)=n^2$ ，其余条件不变，则在新机器上用 t 秒时间能解输入规模为多大的问题？

(3) 若上述算法的计算时间进一步改进为 $T(n)=8$ ，其余条件不变，那么在新机器上用 t 秒时间能解输入规模为多大的问题？

(1)第1-4-(1)题答案:

原计算机能够完成算法的规模为 n

$$\text{则 } t = 3 \times 2^n$$

设在现在的计算机能完成的算法的规模为 n_1

$$\text{则 } 64 \times t = 3 \times 2^{n_1}$$

$$\text{则 } n_1 = \log_2 64 \times \frac{t}{3} = n + 6$$

在新机器问题的规模仅增加了**6**

(2)第1-4-(2)题答案:

原计算机能够完成算法的规模为 n

$$\text{则 } t = n^2$$

设现在计算机能完成的算法规模为 n_2

$$\text{则 } 64 \times t = n_2^2$$

$$\text{则 } n_2 = \sqrt{64 \times t} = 8n$$

在新机器问题的规模扩大到原来的**8倍**

(3)第1-4-(3)题答案:

为常数时间复杂度，与问题规模无关，肯定能完成问题

1-5 硬件厂商 XYZ 公司宣称他们最新研制的微处理器运行速度为其竞争对手 ABC 公司同类产品的 100 倍。对于计算复杂性分别为 n 、 n^2 、 n^3 和 $n!$ 的各算法，若用 ABC 公司的计算机在 1 小时内能解输入规模为 n 的问题，那么用 XYZ 公司的计算机在 1 小时内分别能解输入规模为多大的问题？

第1-5题答案:

(1) 对于线性阶 $f(n) = n$

$$n' = 100 \times n$$

在新机器上能解输入规模为 $100n$ 的问题。

(2) 对于平方阶 $f(n) = n^2$

$$(n')^2 = 100 \times n^2$$

$$\text{对两边开平方, 得 } n' = \sqrt{100} \times n = 10n$$

在新机器上能解输入规模为 $10n$ 的问题。

(3) 对于立方阶 $f(n) = n^3$

$$(n')^3 = 100 \times n^3$$

$$\text{对两边开立方, 得 } n' = \sqrt[3]{100} \times n \approx 4.64n$$

在新机器上能解输入规模约为 $4.64n$ 的问题。

(4) 对于阶乘阶 $f(n) = n!$

阶乘增长极快。

对于阶乘级或指数级算法，硬件性能的提升对可处理规模的增加**极其有限**。

1-6

1-6 对于下列各组函数 $f(n)$ 和 $g(n)$ ，确定 $f(n)=O(g(n))$ 或 $f(n)=\Omega(g(n))$ 或 $f(n)=\theta(g(n))$ ，并简述理由。

(1) $f(n)=\log n^2$; $g(n)=\log n+5$

(2) $f(n)=\log n^2$; $g(n)=\sqrt{n}$

(3) $f(n)=n$; $g(n)=\log^2 n$

(4) $f(n)=n\log n+n$; $g(n)=\log n$

(5) $f(n)=10$; $g(n)=\log 10$

(6) $f(n)=\log^2 n$; $g(n)=\log n$

(7) $f(n)=2^n$; $g(n)=100n^2$

(8) $f(n)=2^n$; $g(n)=3^n$

第1-6题答案:

(1) $f(n) = \log n^2; g(n) = \log n + 5$

$f(n) = 2 \log n$, 而 $g(n) \approx \log n$ 。忽略常数系数和常数项后, 两者都是 $\log n$ 。

$$f(n) = \theta(g(n))。$$

(2) $f(n) = \log n^2; g(n) = \sqrt{n}$

$f(n) = 2 \log n$ 是对数阶, 而 $g(n) = n^{1/2}$ 是幂函数阶。对数函数的增长速度永远慢于任何正幂函数。

$$f(n) = O(g(n))。$$

(3) $f(n) = n; g(n) = \log^2 n$

$f(n)$ 是线性阶, 而 $g(n)$ 是多项式对数阶。线性增长快于对数增长。

$$f(n) = \Omega(g(n))。$$

(4) $f(n) = n \log n + n; g(n) = \log n$

$f(n)$ 的主要项是 $n \log n$, 显然比单纯的 $\log n$ 增长得快得多。

$$f(n) = \Omega(g(n))。$$

(5) $f(n) = 10; g(n) = \log 10$

两者都是常数 ($g(n)$ 约等于 3.32 或 1 或 2.3 取决于底数, 但都是常数)。常数之间互为同阶。

$$f(n) = \theta(g(n))。$$

(6) $f(n) = \log^2 n; g(n) = \log n$

$f(n) = (\log n) \times (\log n)$, 比 $g(n)$ 多了一个增长因子。

$$f(n) = \Omega(g(n))。$$

(7) $f(n) = 2^n; g(n) = 100n^2$

$f(n)$ 是指数阶, 而 $g(n)$ 是平方阶 (幂函数)。指数阶的增长速度远超任何幂函数。

$$f(n) = \Omega(g(n))。$$

(8) $f(n) = 2^n; g(n) = 3^n$

两者虽然都是指数阶, 但底数不同。由于 $2 < 3$, 当 n 增大时, 2^n 会被 3^n 远远甩在后面。

$$f(n) = O(g(n))。$$

1-7 证明 $n! = o(n^n)$ 。

第1-7题答案:

由斯特林公式可知这两个是同阶的 $n! = O(\sqrt{2\pi n}(\frac{n}{e})^n)$ 。

因此

$$\begin{aligned} & \lim_{n \rightarrow \infty} \frac{n!}{n^n} \\ &= \lim_{n \rightarrow \infty} \frac{\sqrt{2\pi n}(\frac{n}{e})^n}{n^n} \\ &= \lim_{n \rightarrow \infty} \frac{e^{\frac{1}{2}\log n} \frac{n^n}{e^n}}{n^n} \\ &= \lim_{n \rightarrow \infty} e^{\frac{\frac{1}{2}\log n}{n}} \\ &= e^{\lim_{n \rightarrow \infty} \frac{\frac{1}{2}\log n}{n}} \\ &= e^0 \\ &= 1 \end{aligned}$$

所以同阶

1-8

1-8 下面的算法段用于确定 n 的初始值。试分析该算法段所需计算时间的上界和下界。

```
while(n>1)
  if(odd(n))
    n = 3*n+1;
  else
    n = n/2;
```

第1-8题答案:

下界:

当初始值 n 本身就是 2 的幂次 (例如 $n = 2^k$) 时, 算法会不断执行 $n = n/2$, 直到变为 1。这个过程需要 $\log_2 n$ 次操作。

因此, 计算时间的下界为 $\Omega(\log n)$ 。

上界：

该算法属于 **3n+1 猜想**。由于目前数学上尚未证明该算法对所有 n 都能停机，因此其计算时间的**上界**在数学上仍是一个未解之谜，无法给出确切的渐近表示。

1-9

1-9 证明：如果一个算法在平均情况下的计算时间复杂性为 $\theta(f(n))$ ，则该算法在最坏情况下所需的计算时间为 $\Omega(f(n))$ 。

第1-9题答案:

设该算法对于规模为 n 的所有输入集合为 S 。

对于每一个输入 $x \in S$ ，算法的执行时间记为 $T(x)$ 。

设平均情况下的时间复杂度为 $T_{avg}(n)$ ，根据题目已知： $T_{avg}(n) = \theta(f(n))$ 。

这意味着存在常数 $c > 0$ 和 n_0 ，使得当 $n \geq n_0$ 时， $T_{avg}(n) \geq c \cdot f(n)$

平均时间 $T_{avg}(n)$ 是所有输入执行时间的平均：

$$T_{avg}(n) = \frac{1}{|S|} \sum_{x \in S} T(x)$$

最坏情况下的时间 $T_{max}(n)$ 是所有输入中执行时间的最大值：

$$T_{max}(n) = \max_{x \in S} \{T(x)\}$$

我们都指导，一组数据的**最大值**必然大于或等于这组数据的**平均值**：

$$\text{即 } T_{max}(n) \geq T_{avg}(n)$$

代入已知条件可知：

由于 $T_{avg}(n) = \Theta(f(n))$ ，则必有 $T_{avg}(n) \geq c \cdot f(n)$ 。

因此：

$$T_{max}(n) \geq T_{avg}(n) \geq c \cdot f(n)$$

因此，如果存在常数 c 和 n_0 ，使得当 $n \geq n_0$ 时， $T(n) \geq c \cdot f(n)$ ，则 $T(n) = \Omega(f(n))$ 。

所以，最坏情况下的计算时间为 $\Omega(f(n))$ 。